МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ

по лабораторной работе №2
по дисциплине «Интеллектуальные методы анализа данных»
Тема: Реализация метода классификации данных с использованием природных временных рядов

Студент гр. 3384 _____ Кашечкин О.В.

Преподаватель _____ Мандрикова Б.С.

Санкт-Петербург
2026

Цель работы.

Построение классификатора данных геофизического мониторинга на примере наблюдений нейтронных мониторов, а именно освоение непараметрического метода прогноза природного временного ряда на основе поиска ближайших соседей в фазовом пространстве, восстановленном по теореме Такенса, включающего подготовку данных, формирование векторов-историй, вычисление расстояний с использованием нескольких метрик, построение прогноза по найденным аналогам и оценку качества полученных предсказаний.

Задание.

Вариант 7. Москва (MOSC), 1-9 августа 2015 г.

Домашнее задание: реализация метода классификации данных с использованием природных временных рядов.

Задачи:

1. Подготовить данные. Скачать данные из источника [https://www.nmdb.eu/nest/] за периоды и станции, указанные по вариантам таблицах. Использовать сданные с одноминутным временным разрешением. При наличии пропусков в данных заполнить их медианным значением (функция median в MatLab).

2. Сформировать векторы-историй. Временной ряд разбить на последовательные блоки (векторы) длиной т, каждый из которых должен содержать т прошлых значений. Последний вектор в ряде будет «тестовой» точкой для поиска соседей. Величину т выбрать равной 180 отсчетов.

3. Найти ближайших соседей. Для «тестового» вектора вычислить расстояние (метрику) до всех остальных векторов-историй (использовать 3 различные метрики). Число соседей (к) определить равным 2-4.

4. Использовать соседей для прогноза. На основе найденных ближайших соседей построить предсказания для будущих значений ряда. При нахождении нескольких схожих отрезков, нужно взять простое среднее следующих за ними значений.

5. Сделать соответствующие выводы, оформить отчет по работе.

Выполнение работы.

Для выполнения задания была написана программа на языке Python с использованием библиотек requests, pandas, numpy и matplotlib. На первом этапе программа проверяет наличие локального файла с данными и, если он отсутствует, формирует запрос к серверу NMDB с параметрами станции MOSC, типа данных corr_for_efficiency, одноминутного разрешения и заданного периода, после чего сохраняет полученный текстовый ответ в локальный файл для повторного использования. Затем выполняется парсинг ASCII-файла, в ходе которого построчно извлекаются дата, время и значение скорости счёта, строки комментариев и заголовки игнорируются, а служебные флаговые значения вида минус 999 и минус 9999 интерпретируются как пропуски. Полученный набор приводится к объекту DataFrame с временным индексом, дубликаты меток времени удаляются, после чего ряд переводится на полную регулярную одноминутную сетку с помощью генерации диапазона дат. Все образовавшиеся при этом пропуски заполняются медианным значением, вычисленным по имеющимся отсчётам, что обеспечивает устойчивость к выбросам и даёт непрерывный ряд, пригодный для дальнейшего анализа. На рассматриваемом наборе данных было загружено ровно 12 960 записей, что соответствует девяти суткам по 1440 минут, диапазон полностью совпал с заданием, а пропусков обнаружено не было, поэтому замена не потребовалась, хотя медиана (около 240.62) была рассчитана и готова к использованию.

На втором этапе формируются векторы-истории. Ряд разбивается на последовательные блоки длиной т = 180 отсчётов с помощью скользящего окна, в результате чего каждый блок представляет собой точку в 180-мерном фазовом пространстве, координатами которой служат последовательные прошлые значения ряда. Последний вектор ряда признаётся тестовым: именно для него в дальнейшем ищутся аналоги в прошлом, а следующее за ним значение считается

неизвестным и подлежащим прогнозу. Чтобы избежать тривиального поиска, когда ближайшим соседом тестового вектора оказывается он сам, сдвинутый всего на одну минуту, из пула соседей исключаются все векторы, перекрывающиеся с тестовым, то есть сдвинутые относительно него менее чем на т отсчётов. Это обеспечивает содержательность поиска и корректность оценки близости.

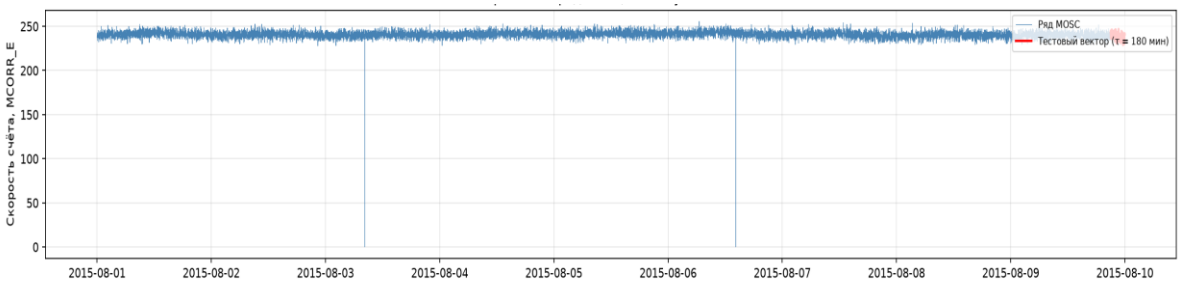


Рисунок 1 – Временной MOSC, 1-9 августа 2015

На третьем этапе для тестового вектора вычисляются расстояния до всех оставшихся векторов-историй с использованием трёх различных метрик: евклидовой, представляющей собой квадратный корень из суммы квадратов покоординатных разностей, манхэттенской, равной сумме модулей покоординатных разностей, и чебышёвской, определяемой максимальным по координатам отклонением. Каждая метрика по-своему реагирует на характер различий: евклидова чувствительна к крупным отклонениям по отдельным координатам, манхэттенская более равномерно учитывает все отклонения и устойчивее к одиночным выбросам, а чебышёвская определяется наихудшим совпадением по одной координате и потому предъявляет наиболее строгие требования к подобию. Число соседей к было выбрано равным трём, что попадает в рекомендованный заданием диапазон от двух до четырёх. Для каждой метрики были найдены три ближайших вектора-истории, их индексы и расстояния до тестового вектора выведены в консоль. Визуализация тестового вектора и найденных соседей на отдельном графике наглядно показала, что все три аналога по своей форме действительно похожи на текущее состояние ряда, что подтверждает применимость теоремы Такенса к рассматриваемым данным и корректность работы алгоритма поиска.



Рисунок 2 – Тестовый вектор и k ближайших соседей (евклидова метрика)

На четвёртом этапе найденные соседи используются для построения прогноза. Для каждого из трёх ближайших векторов берётся значение, следующее непосредственно за ним во временном ряду, и прогноз формируется как простое среднее этих трёх значений. Такой подход является непараметрическим: он не предполагает никакой конкретной формы зависимости, а опирается исключительно на самоподобие ряда и гипотезу о том, что близкие состояния системы приводят к близким последующим значениям. Для тестового вектора все три метрики дали прогнозы в диапазоне примерно от 240 до 243 единиц скорости счёта, что хорошо согласуется со средним уровнем ряда и не содержит аномальных значений. Дополнительно была реализована процедура скользящего контроля walk-forward, в рамках которой прогноз строился не только для последнего вектора, но и для ста последовательных позиций на последних 1500 минутах ряда, при этом для каждой позиции в обучении использовалась только предшествующая информация, что имитирует честные условия реального прогнозирования. Для каждой метрики были вычислены средняя абсолютная ошибка MAE и корень из среднеквадратичной ошибки RMSE. Полученные значения MAE оказались порядка 2.9–3.3 единицы MCORR_E, что составляет менее полутора процентов от среднего уровня сигнала и говорит о приемлемом качестве прогноза для шумных геофизических данных. Дополнительно было исследовано влияние числа соседей на точность: при увеличении k от двух до пяти ошибка MAE монотонно снижалась с примерно 3.38 до 3.06, что объясняется усредняющим действием большего числа

аналогов, подавляющим случайный шум, однако выходить за рекомендованный диапазон к = 2–4 в рамках задания не требовалось.

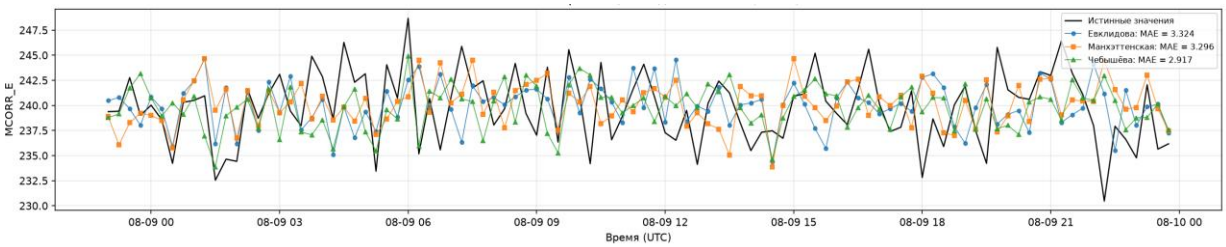


Рисунок 3 – walk-forward прогноз (последние 1500 мин, K=3)

Для визуализации результатов построены три графика выше, расположенные друг под другом. На верхнем графике показан весь исходный временной ряд скорости счёта нейтронного монитора за указанный период, при этом красным цветом выделен тестовый вектор, то есть последние 180 минут наблюдений, для которых ищутся аналоги. На этом же графике видны два характерных глубоких провала до нулевых значений, соответствующих аппаратным сбоям, отмеченным ещё в предыдущей лабораторной работе. На среднем графике в увеличенном масштабе изображены тестовый вектор и три его ближайших соседа, найденных по евклидовой метрике; в легенде для каждого соседа приведены его индекс в ряду и расстояние до тестового вектора. Этот график служит главным визуальным подтверждением работы метода: найденные фрагменты прошлого действительно воспроизводят форму текущего состояния системы с точностью до мелких флуктуаций. На нижнем графике показаны результаты скользящего контроля: чёрной линией изображены истинные значения ряда на последних 1500 минутах, а цветными линиями с маркерами — прогнозы, полученные тремя метриками. В легенде указаны значения MAE для каждой метрики. Из графика видно, что все три прогнозные кривые идут почти параллельно истинной, отслеживая медленную составляющую сигнала и локальный уровень ряда, но не пытаясь воспроизвести резкие одноминутные выбросы, что соответствует теоретическим ожиданиям и природе шумовой компоненты. Наилучшую точность в данном эксперименте показала метрика Чебышёва с MAE около 2.92, что объясняется её меньшей

чувствительностью к отдельным крупным отклонениям внутри вектора по сравнению с евклидовой метрикой.

Вывод.

В ходе лабораторной работы был успешно реализован непараметрический метод прогноза природного временного ряда на основе поиска ближайших соседей в фазовом пространстве, восстановленном по теореме Такенса. Данные нейтронного монитора станции Москва за 1–9 августа 2015 года с одноминутным разрешением были загружены с сервера NMDB, корректно распарсены, приведены к регулярной временной сетке и подготовлены к анализу с заполнением возможных пропусков медианным значением. Ряд был разбит на векторы-истории длиной 180 отсчётов, для последнего (тестового) вектора с использованием трёх различных метрик расстояния найдены три ближайших аналога в прошлом, и на их основе построены прогнозы, которые затем были проверены в режиме скользящего контроля на ста последовательных позициях. Визуализация тестового вектора вместе с найденными соседями наглядно подтвердила гипотезу о самоподобии ряда: в истории наблюдений действительно существуют фрагменты, локально идентичные текущему состоянию системы. Количественная оценка показала, что все три метрики дают сопоставимые результаты со средней абсолютной ошибкой порядка трёх единиц MCORR_E, что составляет менее полутора процентов от среднего уровня сигнала. Наилучшую точность показала метрика Чебышёва, а увеличение числа соседей до пяти дополнительно снижало ошибку за счёт усреднения случайных флуктуаций. Метод ближайших соседей успешно улавливает медленную составляющую сигнала и локальный уровень ряда, однако принципиально не способен предсказывать высокочастотные случайные флуктуации одноминутного масштаба, что является не недостатком конкретной реализации, а фундаментальным ограничением любого метода прогноза шумных природных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

```
import os
import sys
import numpy as np
import pandas as pd
import requests
import matplotlib.pyplot as plt
from numpy.lib.stride_tricks import sliding_window_view

STATION    = "MOSC"
START_DAY, START_MONTH, START_YEAR = 1, 8, 2015
END_DAY,   END_MONTH,   END_YEAR   = 9, 8, 2015
LOCAL_FILE = "mosc_2015_08_01_09.txt"

TAU = 180
K   = 3

def build_url() -> str:
    return (
        "https://www.nmdb.eu/nest/draw_graph.php"
        "?formchk=1"
        f"&stations[]={STATION}"
        "&tabchoice=ori"
        "&dtype=corr_for_efficiency"
        "&tresolution=1"
        "&yunits=0"
        "&date_choice=bydate"

f"&start_day={START_DAY:02d}&start_month={START_MONTH:02d}&start_year={START_
YEAR}"
        "&start_hour=00&start_min=00"

f"&end_day={END_DAY:02d}&end_month={END_MONTH:02d}&end_year={END_YEAR}"
        "&end_hour=23&end_min=59"
```

```
        "&output=ascii&force=1"
    )

def download_data() -> str:
    if os.path.exists(LOCAL_FILE):
        print(f"[i] Используется локальный файл: {LOCAL_FILE}")
        with open(LOCAL_FILE, "r", encoding="utf-8", errors="ignore") as f:
            return f.read()
    print("[i] Скачивание данных с NMDB...")
    r = requests.get(build_url(), headers={"User-Agent": "Mozilla/5.0"}, timeout=120)
    r.raise_for_status()
    with open(LOCAL_FILE, "w", encoding="utf-8") as f:
        f.write(r.text)
    print(f"[i] Данные сохранены в {LOCAL_FILE}")
    return r.text

def parse_data(text: str) -> pd.DataFrame:
    rec = []
    sentinels = {-999.0, -9999.0, -99999.0, 9999.0, 99999.0}
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        if "start_date_time" in line:
            continue
        if ";" not in line:
            continue
        dt_str, val_str = line.split(";", 1)
        val_str = val_str.strip()
        if val_str == "" or val_str.lower() in ("null", "nan", "na", "-"):
            v = np.nan
        else:
            try:
                v = float(val_str)
            except ValueError:
```

```
                continue
            if v in sentinels:
                v = np.nan
        try:
            dt = pd.to_datetime(dt_str.strip())
        except Exception:
            continue
        rec.append((dt, v))
    if not rec:
        sys.exit("Не удалось распарсить данные.")
    df = pd.DataFrame(rec, columns=["datetime", "value"]).set_index("datetime").sort_index()
    df = df[~df.index.duplicated(keep="first")]
    return df

def fill_gaps(df: pd.DataFrame):
    full_idx = pd.date_range(
        df.index.min().floor("min"),
        df.index.max().ceil("min"),
        freq="1min",
    )
    s = df["value"].reindex(full_idx)
    n_gaps = int(s.isna().sum())
    med = float(s.median())
    s = s.fillna(med)
    return s, med, n_gaps

def dist_euclidean(H, v):  return np.sqrt(np.sum((H - v) ** 2, axis=1))
def dist_manhattan(H, v):  return np.sum(np.abs(H - v), axis=1)
def dist_chebyshev(H, v):  return np.max(np.abs(H - v), axis=1)

METRICS = {
    "Евклидова":    dist_euclidean,
    "Манхэттенская": dist_manhattan,
    "Чебышёва":     dist_chebyshev,
```

```
}

def build_histories(values: np.ndarray, tau: int) -> np.ndarray:
    """Матрица (N-tau+1, tau): строка i = values[i:i+tau]."""
    return sliding_window_view(values, tau)

def predict_last(values: np.ndarray, tau: int, k: int, metric_fn):
    N = len(values)
    H = build_histories(values, tau)
    cutoff = N - 2 * tau
    pool = H[:cutoff]
    test_vec = values[-tau:]
    dists = metric_fn(pool, test_vec)
    order = np.argsort(dists)[:k]
    next_vals = values[order + tau]
    pred = float(np.mean(next_vals))
    return pred, order, dists, test_vec

def walk_forward(values: np.ndarray, tau: int, k: int, metric_fn,
                 n_test: int = 1500, step: int = 15):
    N = len(values)
    H = build_histories(values, tau)
    idx_list, pred_list, true_list = [], [], []
    t_start = N - n_test
    for t in range(t_start, N, step):
        cutoff = t - 2 * tau
        if cutoff < 10:
            continue
        pool = H[:cutoff]
        test_vec = values[t - tau:t]
        dists = metric_fn(pool, test_vec)
        order = np.argsort(dists)[:k]
        next_vals = values[order + tau]
        pred_list.append(float(np.mean(next_vals)))
        true_list.append(float(values[t]))
```

```
        idx_list.append(t)
    return np.array(idx_list), np.array(pred_list), np.array(true_list)

def main():
    print("=" * 66)
    print(" Вариант 7: MOSC, 01–09 августа 2015, k-NN прогноз")
    print("=" * 66)

    text = download_data()
    df = parse_data(text)
    print(f"[i] Загружено записей: {len(df)}")
    print(f"[i] Диапазон: {df.index.min()} — {df.index.max()}")

    series, med, n_gaps = fill_gaps(df)
    print(f"[i] Регулярная 1-мин сетка: {len(series)} точек")
    print(f"[i] Пропусков заполнено медианой ({med:.3f}): {n_gaps}")

    values = series.values.astype(float)
    print(f"[i] τ = {TAU}, K = {K}")

    print("\nПрогноз следующего значения для тестового вектора")
    results = {}
    for name, fn in METRICS.items():
        pred, order, dists, test_vec = predict_last(values, TAU, K, fn)
        results[name] = {"pred": pred, "order": order, "dists": dists, "test_vec": test_vec}
        d_str = ", ".join(f"{dists[i]:.2f}" for i in order)
        print(f"  {name:15s}: прогноз = {pred:8.3f}  |  "
              f"индексы соседей = {list(order)}  |  d = [{d_str}]")

    print("\nWalk-forward оценка на последних 1500 мин")
    wf = {}
    for name, fn in METRICS.items():
        idx, pred, true = walk_forward(values, TAU, K, fn, n_test=1500, step=15)
        mae = float(np.mean(np.abs(pred - true)))
        rmse = float(np.sqrt(np.mean((pred - true) ** 2)))
```

```
    wf[name] = {"idx": idx, "pred": pred, "true": true, "mae": mae, "rmse": rmse}
    print(f"  {name:15s}: MAE = {mae:.4f}   RMSE = {rmse:.4f}   "
          f"точек = {len(idx)}")

print("\nВлияние числа соседей K (евклидова метрика)")
for kk in (2, 3, 4, 5):
    idx, pred, true = walk_forward(values, TAU, kk, dist_euclidean,
                                   n_test=1500, step=15)
    mae = float(np.mean(np.abs(pred - true)))
    print(f"  K = {kk}: MAE = {mae:.4f}")

fig = plt.figure(figsize=(15, 11))

ax1 = plt.subplot2grid((3, 2), (0, 0), colspan=2)
ax1.plot(series.index, values, color="steelblue", lw=0.5, label="Ряд MOSC")
ax1.plot(series.index[-TAU:], values[-TAU:], color="red", lw=1.8,
         label=f"Тестовый вектор (τ = {TAU} мин)")
ax1.set_title("Временной ряд MOSC, 1–9 августа 2015")
ax1.set_ylabel("Скорость счёта, MCORR_E")
ax1.grid(True, alpha=0.3)
ax1.legend(loc="upper right", fontsize=8)

ax2 = plt.subplot2grid((3, 2), (1, 0), colspan=2)
ref = results["Евклидова"]
for rank, i in enumerate(ref["order"]):
    ax2.plot(np.arange(TAU), values[i:i + TAU], lw=1.0,
             label=f"Сосед #{rank+1} (i={i}, d={ref['dists'][i]:.2f})")
ax2.plot(np.arange(TAU), ref["test_vec"], color="red", lw=2.0,
         label="Тестовый вектор")
ax2.set_title("Тестовый вектор и k ближайших соседей (евклидова метрика)")
ax2.set_xlabel(f"Смещение внутри вектора (τ = {TAU} мин)")
ax2.set_ylabel("MCORR_E")
ax2.grid(True, alpha=0.3)
ax2.legend(fontsize=8)
```

```
    ax3 = plt.subplot2grid((3, 2), (2, 0), colspan=2)
    wfe = wf["Евклидова"]
    times = series.index[wfe["idx"]]
    ax3.plot(times, wfe["true"], color="black", lw=1.2, label="Истинные значения")
    for name, marker in zip(METRICS.keys(), ["o", "s", "^"]):
        w = wf[name]
        ax3.plot(series.index[w["idx"]], w["pred"],
                 marker=marker, ms=3.5, lw=0.8, alpha=0.85,
                 label=f"{name}: MAE = {w['mae']:.3f}")
    ax3.set_title("Walk-forward прогноз (последние 1500 мин, K = 3)")
    ax3.set_xlabel("Время (UTC)")
    ax3.set_ylabel("MCORR_E")
    ax3.grid(True, alpha=0.3)
    ax3.legend(fontsize=8, loc="upper right")

    plt.tight_layout()
    out = "mosc_knn.png"
    plt.savefig(out, dpi=150)
    print(f"\n[i] Графики сохранены в {out}")
    plt.show()

if __name__ == "__main__":
    main()
```